

Examples of Review/Retrospective Patterns

Conversation Log | June 27, 2026

USER

create examples for the Review/retrospective patterns

CLAUDE

Here's one example for each of the two patterns under Review/Retrospective — Pre-Mortem and Retrospective/Postmortem.

Pre-Mortem

Before starting work, ask “assume this fails — why?” to surface risks while there's still time to change course, rather than discovering them after the fact.

“I'm about to approve the plan for adding --regex support to TextFinder1. Before I approve it, do a pre-mortem: assume we implemented this exactly as planned and it caused problems three months from now. What's the most likely reason, and what in the current plan should change to prevent it?”

The model might respond: the most likely failure isn't a bug in the regex matching itself, but scope creep at the boundary — once --regex exists, the natural next request is “can it also support look-ahead” or “can it highlight capture groups in output,” and the current plan doesn't say where that line is. Three months from now, the feature could be half-supporting things it was never designed for because nothing in the plan explicitly closed that door. Fix: add an explicit “Out of Scope” note to the plan now, before implementation, rather than relitigating it feature-request by feature-request later.

Why it matters: the failure mode identified here isn't a coding mistake at all — pre-mortems are good at surfacing process and scope risks that a code review or test suite would never catch, because nothing is broken yet; the plan is just incomplete in a way that won't bite until later.

Retrospective / Postmortem

After the work is done, review what actually happened against what was planned — not to assign blame, but to extract a concrete change for next time.

“We just finished implementing --regex for TextFinder1. Do a retrospective: where did the actual implementation deviate from the original plan, what caused each deviation, and what should change in how we plan features like this going forward?”

The model might report: the plan assumed the regex crate's `Regex::new()` would be the only place errors needed handling, but in practice invalid regex syntax also needed to be caught earlier, at CLI-arg-parsing time, to give a cleaner error message before any file traversal started — something the plan didn't anticipate because it focused on the matching logic and not the CLI layer. Going forward: plans for new flags should explicitly call out “where in the pipeline can this fail” as its own section, not just “how does the feature work once inputs are valid.”

Why it's distinct from Pre-Mortem: Pre-Mortem reasons about a hypothetical failure before anything exists, using imagination and pattern-matching to past projects; Retrospective reasons about what actually happened, using the real deviations as evidence — it's strictly more reliable, but only available after the cost of any mistakes has already been paid.

The two bookend the same work from opposite ends: Pre-Mortem tries to buy down risk before it's incurred, while Retrospective makes sure whatever risk did materialize gets converted into a lesson — for instance, feeding back into your constitution.md or template structure — rather than just being fixed and forgotten.